Recitation 7 – Authentication, Authorization, and Token-Based Security

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. Authentication vs Authorization
2. Token-Based Authentication
3. Types of Tokens
4. JWT (JSON Web Tokens)
5. OAuth 2.0
6. OpenID Connect (OIDC)
7. Using Tokens with REST APIs

Today

- Authentication vs Authorization
- Token-Based Authentication
- Types of Tokens (Access, Refresh, Identity)
- JWT (JSON Web Tokens) - Deep Dive
- OAuth 2.0 - Authorization Framework
- OpenID Connect (OIDC) - Identity Layer
- Using Tokens with REST APIs
- Security Best Practices

Why Security for REST APIs?

The Problem

REST APIs are **stateless** - each request is independent. How do we:

- Know **who** is making the request?
- Control **what** they can access?

Real-World Scenarios:

- User accessing their own profile data
- Admin deleting a user account
- Mobile app fetching private messages
- Third-party app accessing your Google Calendar

Solution: Authentication and Authorization mechanisms with tokens

Authentication vs Authorization

Authentication (AuthN)

"Who are you?"

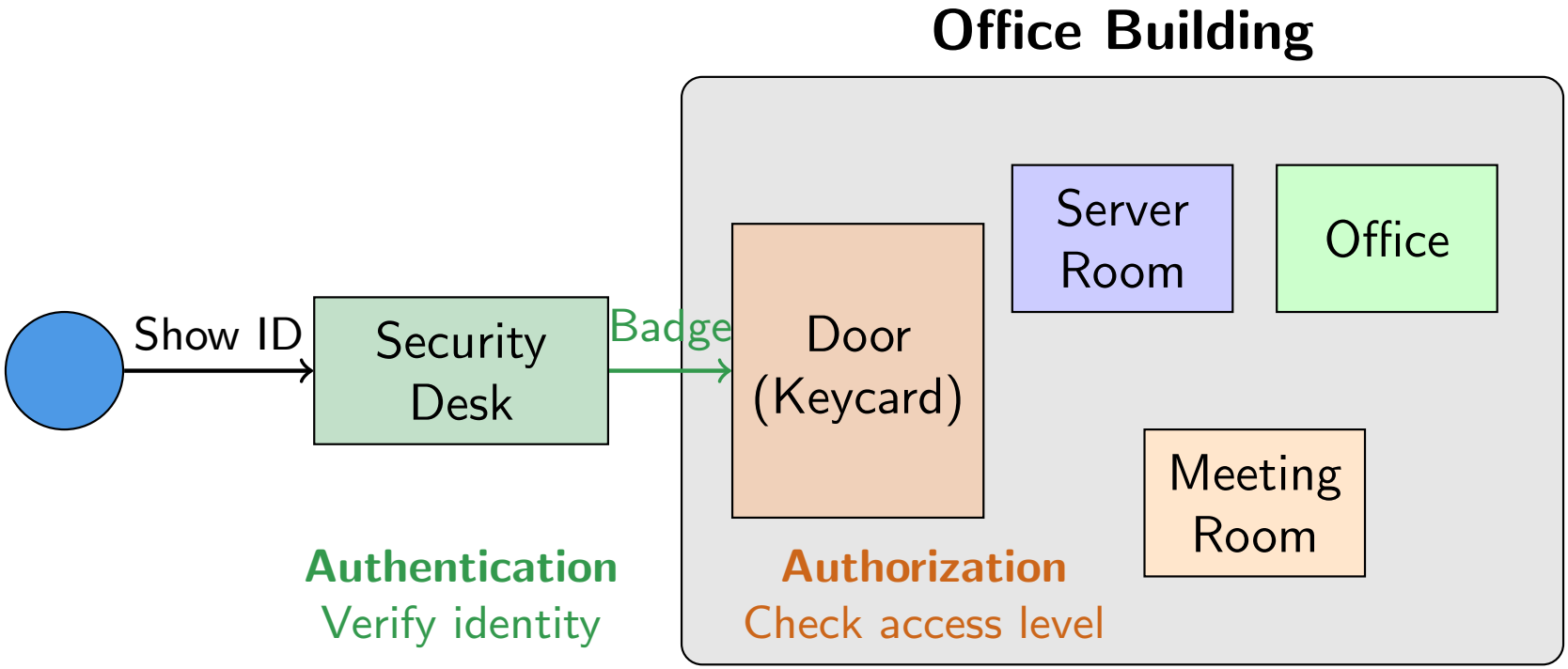
- Verifying **identity**
- Proving you are who you claim to be
- Examples:
  - Username + Password
  - Fingerprint scan
  - ID card check

Authorization (AuthZ)

"What can you do?"

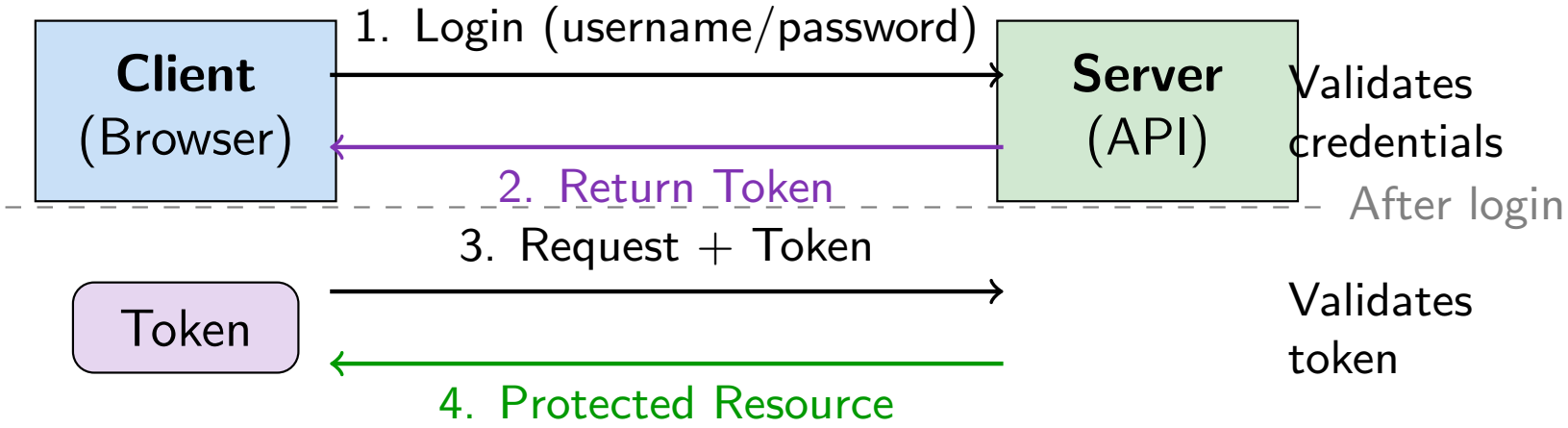
- Verifying **permissions**
- Determining access rights
- Examples:
  - Admin vs regular user
  - Read-only vs read-write
  - Access to specific resources

Authentication vs Authorization - Analogy



Different rooms =
Different permissions

The Authentication Flow



Stateless: Server doesn't store session. Each request carries its own authentication (the token).

Why Tokens?

Session-Based (Traditional)

- Server stores session data
- Session ID in cookie
- **Stateful** - server memory required
- Hard to scale horizontally
- Tied to single domain

Token-Based (Modern)

- Client stores token
- Token in header/cookie
- **Stateless** - no server storage
- Easy to scale (any server can validate)
- Works across domains/services

What is a Token?

Token Definition

A **token** is a piece of data that represents a user's authentication state and/or permissions. It's issued by a trusted authority and can be verified without contacting that authority again.

Token Characteristics:

- **Self-contained** - Contains all necessary information
- **Signed** - Can be verified for authenticity
- **Expirable** - Has a limited lifetime
- **Portable** - Can be used across services

Analogy:

- Like a concert ticket - contains info about the event, seat, validity
- Security can verify it's genuine without calling the box office

Types of Tokens - Overview



Used for every API request

Used when access token expires

Used to get user profile

Access Token

Access Token

The credential used to access protected resources. Sent with every API request.

Characteristics:

- **Short-lived** - typically 5 minutes to 1 hour
- **Sent in Authorization header** - `Bearer <token>`
- **Contains permissions/scopes** - what the token allows
- **Should never be stored long-term** - security risk

Why short-lived?

- If stolen, limited time window for misuse
- Forces regular re-validation
- Permissions can be updated on refresh

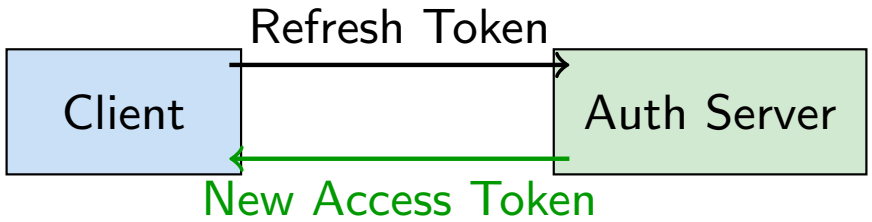
Refresh Token

Refresh Token

A credential used to obtain new access tokens without requiring the user to re-authenticate.

Characteristics:

- **Long-lived** - days to months
- **Never sent to resource APIs** - only to auth server
- **Stored securely** - often in HttpOnly cookie, session or local storage (beyond our scope)
- **Can be revoked** - logout invalidates it



Identity Token (ID Token)

ID Token (OIDC)

Contains verified information about the authenticated user. Specific to OpenID Connect.

Characteristics:

- **Always a JWT** - standardized format
- **Contains user claims** - name, email, picture, etc.
- **For the client application** - not for API access
- **Proves authentication occurred** - who logged in, when, how

Common ID Token Claims:

- `sub` - Subject (unique user ID)
- `name` - User's full name
- `email` - User's email address
- `picture` - URL to profile picture

Token Types - Comparison

Property	Access Token	Refresh Token	ID Token
Purpose	API access	Get new tokens	User info
Lifetime	Short (mins-hrs)	Long (days-months)	Short
Format	Any (often JWT)	Opaque string	Always JWT
Sent to	Resource Server	Auth Server only	Client only
Contains	Scopes/permissions	Reference only	User claims
Protocol	OAuth 2.0	OAuth 2.0	OIDC

Remember: Access tokens are for **authorization** (what you can do). ID tokens are for **authentication** (who you are).

What is JWT?

JSON Web Token (JWT)

JWT (pronounced "jot") is an open standard (RFC 7519) for securely transmitting information between parties as a JSON object. The information is digitally signed and can be verified.

Key Properties:

- **Compact** - Can be sent via URL, header, or body
- **Self-contained** - Contains all user info needed
- **Signed** - Integrity can be verified (tampering detected)
- **Optionally encrypted** - Can hide contents (JWE)

Common Use: JWTs are the most popular format for access tokens and are required for ID tokens in OIDC.

JWT Header

Header (Decoded)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Header Fields:

- `alg` - Signing algorithm
  - `HS256` - HMAC with SHA-256 (symmetric key)
  - `RS256` - RSA with SHA-256 (asymmetric keys)
  - `ES256` - ECDSA with SHA-256
- `typ` - Token type (usually "JWT")

JWT Payload - Claims

Payload (Decoded)

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "email": "john@example.com",
  "role": "admin",
  "iat": 1516239022,
  "exp": 1516242622,
  "aud": "my-api.example.com",
  "iss": "auth.example.com"
}
```

Claims are statements about the user and metadata.

JWT Claims - Types

Three types of claims:

1. **Registered Claims** (standardized, recommended)

Claim	Name	Description
`iss`	Issuer	Who created the token
`sub`	Subject	Who the token is about (user ID)
`aud`	Audience	Who the token is intended for
`exp`	Expiration	When the token expires (Unix time)
`iat`	Issued At	When the token was created
`nbf`	Not Before	Token not valid before this time

2. **Public Claims** - Registered in IANA JWT registry (e.g., `email`, `name`)
3. **Private Claims** - Custom claims agreed upon by parties (e.g., `role`, `tenant_id`)

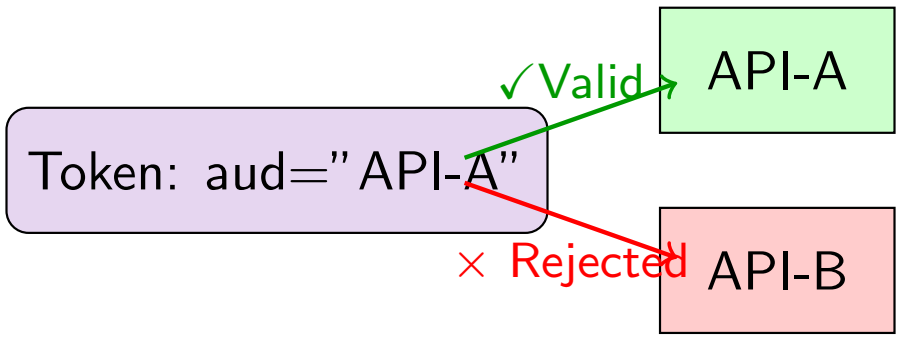
Understanding Audience (aud)

Audience Claim

The aud (audience) claim identifies the **recipients** that the JWT is intended for. The recipient MUST verify it matches their identifier.

Why is this important?

- Prevents token misuse across different APIs
- A token for `api.example.com` shouldn't work on `other-api.com`



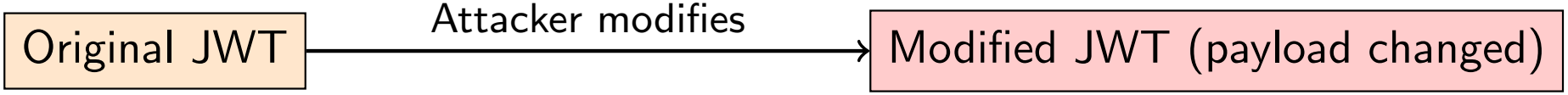
JWT Signature

How the Signature is Created

```
HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),
secret
)
```

Signature Purpose:

- **Integrity** - Detects if token was modified
- **Authenticity** - Proves token came from trusted source
- **NOT encryption** - Payload is still readable



Signature won't match! Server rejects token

HMAC vs RSA Signing

HMAC (HS256)

Symmetric - Same secret for signing and verification

- Fast and simple
- Secret must be shared
- Good for single-service
- Both parties need the secret

RSA (RS256)

Asymmetric - Private key signs, public key verifies

- More secure for distributed systems
- Public key can be shared freely
- Good for microservices
- Auth server keeps private key

What is OAuth 2.0?

OAuth 2.0

OAuth 2.0 is an **authorization framework** that enables applications to obtain limited access to user accounts on third-party services, without exposing passwords.

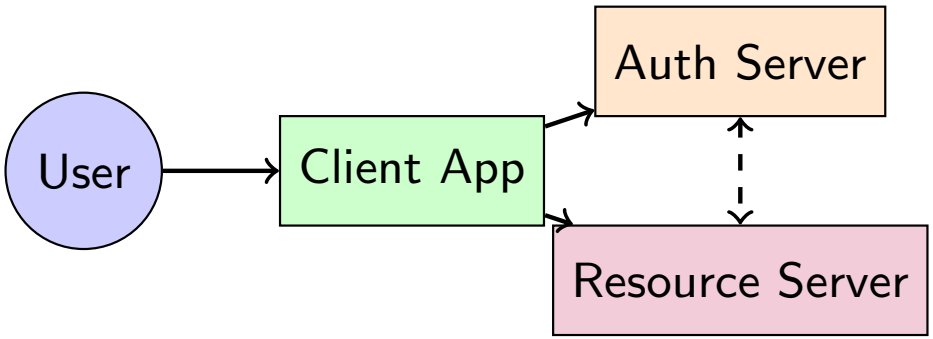
The Problem OAuth Solves:

- You want a third-party app to access your Google Calendar
- But you don't want to give that app your Google password
- OAuth allows granting **limited access** without sharing credentials

Key Point: OAuth is about **authorization** (what the app can access), not authentication (proving who you are). That's why OIDC was created on top of it.

OAuth 2.0 Terminology

Term	Description
Resource Owner	The user who owns the data (you)
Client	The application requesting access (third-party app)
Authorization Server	Issues tokens after user consent (Google's auth)
Resource Server	Hosts the protected data/API (Google Calendar API)
Scope	Specific permissions requested (e.g., "read:calendar")

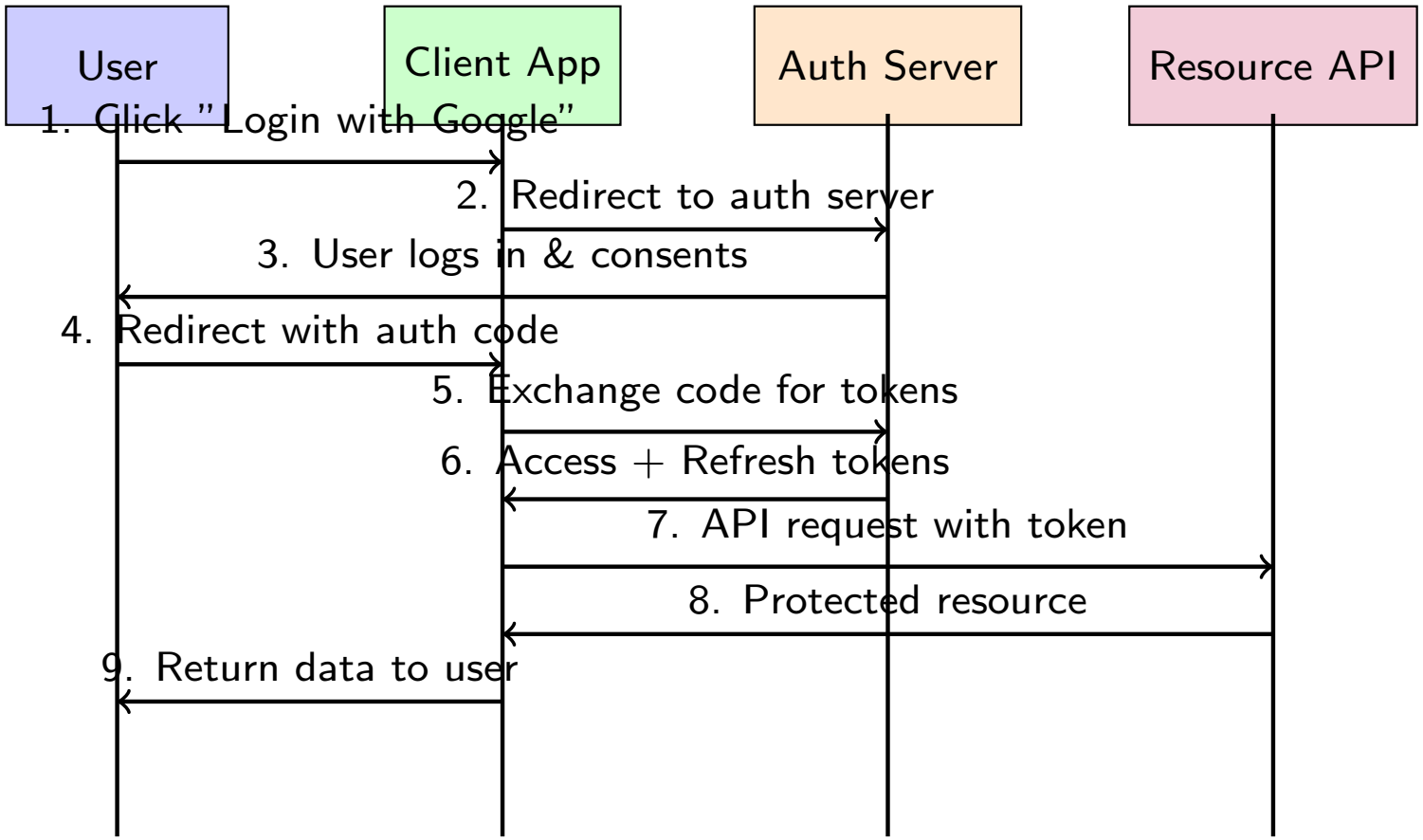


OAuth 2.0 Grant Types

Different flows for different scenarios:

1. **Authorization Code** (most common and the one we will focus on)
   - For web apps with a backend
   - Most secure - uses code exchange
2. **Authorization Code + PKCE** (out of scope)
   - For mobile/SPA apps (no client secret)
   - Adds code verifier for security
3. **Client Credentials** (we will talk about)
   - Machine-to-machine (no user involved)
   - Service accounts, background jobs
4. ~~**Implicit**~~ (deprecated) (out of scope)
   - Was for SPAs - now use PKCE instead
5. ~~**Password**~~ (deprecated) (out of scope)
   - Direct password sharing - avoid!

Authorization Code Flow



Authorization Code Flow - Step by Step

Step 2 - Authorization Request:

```
GET /authorize?
  response_type=code
  &client_id=my-app-id
  &redirect_uri=https://myapp.com/callback
  &scope=read:calendar
  &state=xyz123
Host: auth.google.com
```

Step 4 - Callback with Code:

```
GET /callback?code=AUTH_CODE_HERE&state=xyz123
Host: myapp.com
```

Authorization Code Flow - Token Exchange

Step 5 - Exchange Code for Tokens:

```
POST /token HTTP/1.1
Host: auth.google.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code
&code=AUTH_CODE_HERE
&redirect_uri=https://myapp.com/callback
&client_id=my-app-id
&client_secret=my-app-secret
```

Authorization Code Flow - Token Exchange

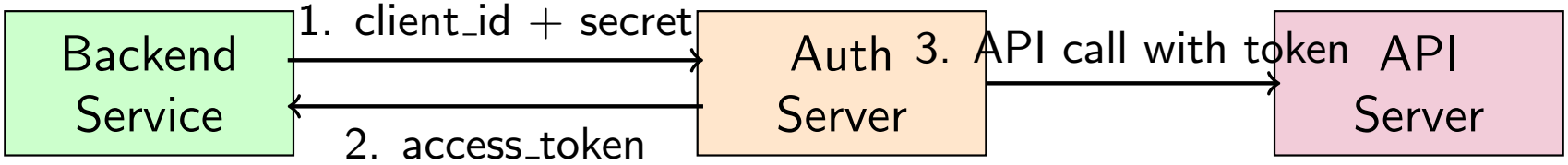
Step 6 - Token Response:

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "dGhpcyBpcyBhIHJlZnJl..."
}
```

Client Credentials Flow

Machine-to-Machine Authentication

Used when there is **no user involved** - the application itself is the resource owner.



Use Cases:

- Microservice-to-microservice communication
- Cron jobs accessing APIs
- Backend data synchronization

What is OpenID Connect?

OpenID Connect (OIDC)

OIDC is an **identity layer** built on top of OAuth 2.0. It adds authentication capabilities to OAuth's authorization framework.

OpenID Connect - Authentication ("Who are you?")

↑ builds on

OAuth 2.0 - Authorization ("What can you access?")

What is OpenID Connect?

What OIDC Adds:

- **ID Token** - JWT containing user identity
- **UserInfo endpoint** - API to get user profile
- **Standardized scopes** - `openid`, `profile`, `email`

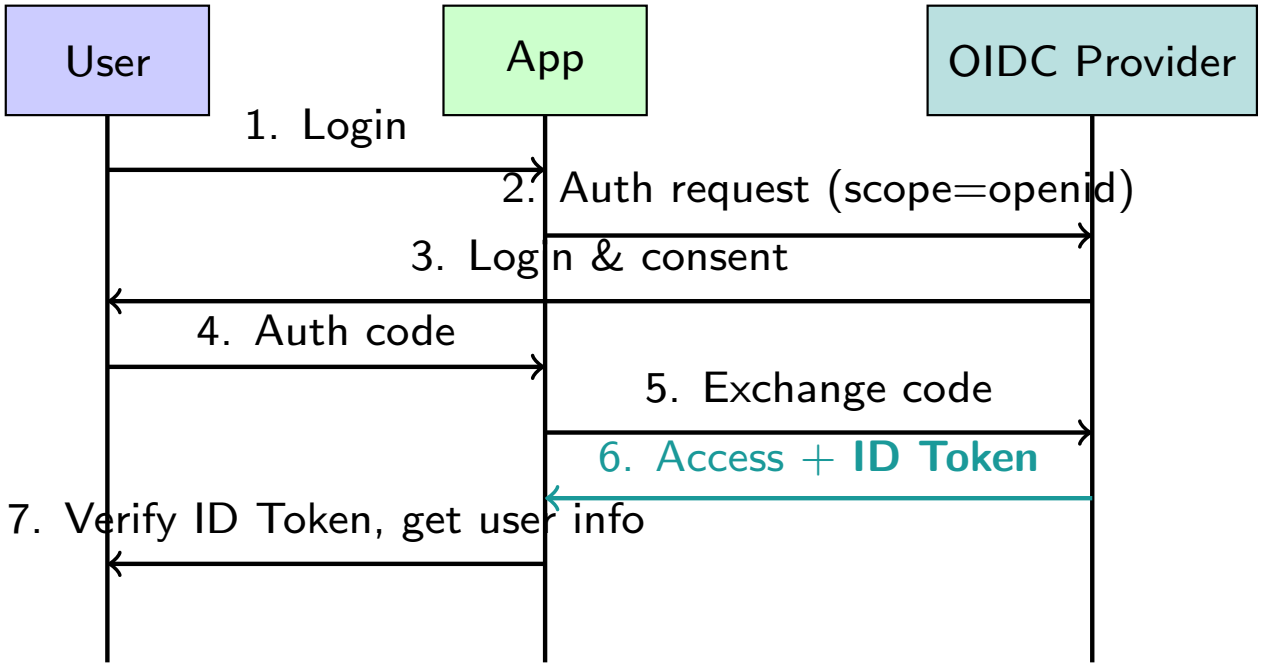
OAuth 2.0 vs OpenID Connect

Feature	**OAuth 2.0**	**OIDC**
Primary Purpose	Authorization	Authentication
Question Answered	"What can you access?"	"Who are you?"
Access Token	✓Yes	✓Yes
ID Token	× No	✓Yes
User Info	Not standardized	Standardized endpoint
Use Case	API access	Login / SSO

Simple Rule:

- Need to know **who the user is**? Use OIDC
- Only need to **access an API** on user's behalf? OAuth is enough

OIDC Flow



The key difference: Response includes an **ID Token** (JWT) with user identity claims!

OIDC ID Token Example

Decoded ID Token Payload


```
{
  "iss": "https://accounts.google.com",
  "sub": "110169484474386276334",
  "aud": "my-app.apps.googleusercontent.com",
  "exp": 1516242622,
  "iat": 1516239022,
  "nonce": "abc123",
  "name": "John Doe",
  "email": "john@gmail.com",
  "email_verified": true,
  "picture": "https://..."
}
```

Common OIDC Providers

Provider	**Use Case**
Google	Consumer apps, GSuite integration
Microsoft (Azure AD)	Enterprise, Office 365 integration
Apple	iOS apps (Sign in with Apple)
Okta	Enterprise identity management
Auth0	Developer-friendly, multi-provider
Keycloak	Self-hosted, open-source

Single Sign-On (SSO): OIDC enables users to log in once and access multiple applications - "Login with Google" across different services.

Sending Tokens in HTTP Requests

Authorization Header (Recommended)

```
GET /api/users/me HTTP/1.1
Host: api.example.com
Authorization: Bearer
    eyJhbGciOiJSUzI1NiIsInR5cCI6...
```

The Bearer Token Scheme:

- `Authorization: Bearer <token>`
- "Bearer" means "whoever bears this token can access"
- Token is sent with every API request
- Server validates token before processing request

API Request with Token - Complete Example

Request:

```
GET /api/users/123/orders HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9
    ...
Content-Type: application/json
```

Success Response (200 OK):

```
{
  "orders": [
    {"id": "ORD-001", "total": 99.99, "status": "shipped"},
    {"id": "ORD-002", "total": 49.99, "status": "pending"}
  ]
}
```

Authentication Error Responses

401 Unauthorized

No valid token provided

```
{
  "error": "unauthorized",
  "message": "Missing or
    invalid token"
}
```

Causes:

- No token sent
- Token expired
- Invalid signature

403 Forbidden

Valid token, but insufficient permissions

```
{
  "error": "forbidden",
  "message": "Insufficient
    permissions"
}
```

Causes:

- Missing required scope
- Role not authorized
- Resource access denied

Token Validation on Server

Server-side validation steps:

1. **Extract token** from Authorization header
2. **Verify signature** - Ensure token wasn't tampered with
   - Use public key (RS256) or shared secret (HS256)
3. **Check expiration** - Is `exp` claim in the future?
4. **Validate audience** - Does `aud` match this API?
5. **Validate issuer** - Is `iss` a trusted auth server?
6. **Check permissions** - Does token have required scopes?
7. **Process request** - If all checks pass

Stateless Validation: All information needed is in the JWT itself - no database lookup required!